Documentación del proyecto: editor de video

Editor de video **no destructivo** para Windows, escrito en C++20 con MFC y las bibliotecas de FFmpeg. Es un proyecto académico con arquitectura **MVC estricta**: los archivos originales nunca se modifican, y el proyecto solo guarda referencias a tramos de esos archivos.

Este documento explica qué hace la aplicación, cómo se usa y cómo está construida por dentro.

Contenido

1. Funciones
2. Puesta en marcha
3. Guía de uso
4. Arquitectura
5. Estructura del repositorio
6. Capa Model
7. Capa Engine
8. Capa Controller
9. Capa View
10. Hilos y concurrencia
11. Formato del proyecto (JSON)
12. Pruebas
13. Especificaciones (OpenSpec)
14. Limitaciones conocidas

1. Funciones

Área	Qué permite
Importación	Videos y audios (el diálogo filtra mp4, mkv, mov, avi, webm, m4v, mp3, wav, aac y m4a). Los archivos inválidos se rechazan con un mensaje que nombra el archivo y la causa.
Timeline	Varias pistas; mover, dividir, recortar y eliminar clips (dejando el hueco o cerrándolo); selección múltiple, mover sobrescribiendo y pegado (imán) según el zoom.
Audio	Cada pista de video tiene su pista de audio (`AV1` , `AV2` ...) con el audio vinculado de sus clips. También hay pistas generales (`A1` , `A2` ...), mute por pista y mezcla. El audio se puede desvincular.
Efectos	Por clip y en orden: escala de grises, brillo/contraste, desenfoque y fade in/out.
Transiciones	Fundido, barrido y deslizamiento (izquierda o derecha) y apertura circular, con crossfade de audio cuando los dos clips tienen audio vinculado.
Composición	Varias pistas de video superpuestas, con posición, escala y opacidad por clip (picture-in-picture).
Preview	Reproducción en tiempo real con audio, cursor arrastrable y actualización al editar.

Área	Qué permite
Exportación	mp4, mkv o mov, con códec, bitrate y resolución configurables. Corre en segundo plano, con progreso y cancelación.
Persistencia	Proyectos en JSON. Los medios que faltan quedan *offline* y se avisa si hay cambios sin guardar.

2. Puesta en marcha

Requisitos

- Windows 10/11 x64. Es la única plataforma configurada, y la app depende de MFC y XAudio2.
- Visual Studio 2026 con **Desarrollo para el escritorio con C++** y el componente **MFC de C++** (toolset v145).
- Un build de FFmpeg 9 con `include\` y con los `.lib`, las DLL y `ffmpeg.exe` en `bin\`. La ruta por defecto es `E:\PrograSistemas\FFmpeg_9\msvc`. Para usar otra, cambia la macro `FFmpegRoot` en `props/FFmpeg.props`. Si la ruta es incorrecta, el build se detiene con un error claro.

`nlohmann/json` y `doctest` ya vienen en `third_party/`.

Compilar y ejecutar

1. Abre `VideoEditor.slnx`.
2. Elige `Debug | x64` o `Release | x64`.
3. Establece **VideoEditor** como proyecto de inicio y pulsa **F5**.

Todos los ejecutables se generan en `bin\<Configuración>\`, y las DLL de FFmpeg se copian allí al terminar el build.

3. Guía de uso

Flujo básico

1. **Importar** (`Ctrl+I`): el video va a la pista de video seleccionada, con su audio vinculado, a continuación del último clip. El primer video del proyecto define el lienzo (resolución y fps).
2. **Editar** en el timeline y ajustar los clips en el panel de propiedades.
3. **Revisar** en el preview con la barra espaciadora.
4. **Guardar** (`Ctrl+S`) y **exportar** (`Ctrl+E`).

Atajos de teclado

Atajo	Acción
`Ctrl+N` / `Ctrl+O` / `Ctrl+S`	Nuevo, abrir y guardar proyecto
`Ctrl+I` / `Ctrl+E`	Importar y exportar
`Espacio`	Reproducir o pausar
`S`	Dividir en el cursor

Atajo	Acción
`Supr`	Eliminar y dejar el hueco
`Shift+Supr`	Eliminar y cerrar el hueco (ripple)

Mouse en el timeline

Gesto	Acción
Clic o arrastre en la regla	Mover el cursor
Arrastrar un clip	Moverlo, también a otra pista del mismo tipo
Arrastrar el borde de un clip	Recortarlo (trim)
`Ctrl` +clic	Agregar o quitar un clip de la selección
Arrastrar un clip de la selección	Mover todo el grupo
Soltar con `Shift`	Sobrescribir: recorta los clips que quedan debajo
Clic derecho en un clip	Dividir, eliminar, desvincular audio o agregar transición
Clic en el rombo de una transición	Seleccionarla para editarla en el panel
Clic derecho en el rombo	Quitar la transición
Botón `M` de una pista de audio	Silenciarla o reactivarla
Rueda / `Ctrl` +rueda	Desplazamiento horizontal / zoom alrededor del mouse

Panel de propiedades

Cambia según la selección:

- **Clip:** puntos de entrada y salida numéricos, lista de efectos y, en pistas de video, posición, escala y opacidad.
- **Transición:** estilo, dirección y duración.
- **Sin selección:** panel del medio.

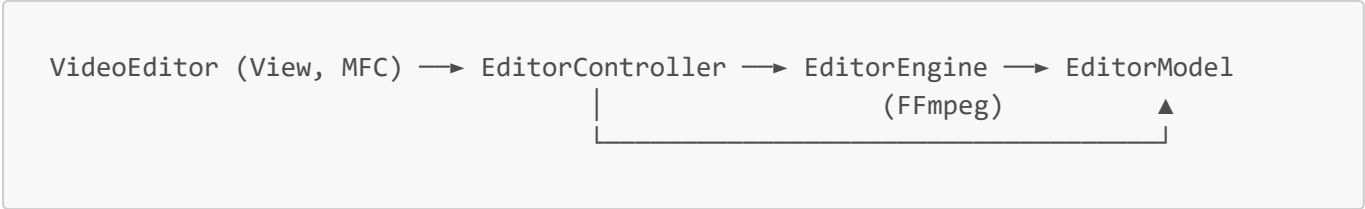
Exportar

El diálogo solo ofrece los encoders que abren en ese equipo y que el contenedor admite. Antes de empezar se valida lo siguiente:

- El timeline no está vacío.
- La salida no es uno de los archivos fuente.
- La resolución es par y está entre 16 y 7680 px.
- El bitrate de video está entre 100 y 100000 kbps, y el de audio entre 32 y 512 kbps.

Si se cancela o falla, el archivo parcial se borra. Mientras se exporta, el controller rechaza las acciones que modifican el proyecto.

4. Arquitectura



Proyecto	Capa	Responsabilidad	No puede usar
`src/EditorModel`	Model	Datos y reglas de edición, serialización JSON	FFmpeg, MFC, Windows
`src/EditorEngine`	Engine	Análisis, decodificación, filtros, preview, exportación	MFC
`src/EditorController`	Controller	Traduce acciones del usuario y guarda el estado de edición	FFmpeg, MFC
`src/VideoEditor`	View	Ventanas, diálogos y dibujo	FFmpeg, modificar el modelo directamente
`tests/EditorTests`	Tests	Pruebas con doctest	MFC

Cómo se protegen las fronteras

Frontera	Protección
Model y Controller sin FFmpeg	Del compilador: esos proyectos no importan `FFmpeg.props`, así que el include falla con `C1083`.
MFC solo en la View	Del compilador: solo VideoEditor tiene `UseOfMfc=Dynamic`; en las demás capas `<afxwin.h>` falla con `C1189`.
Headers públicos del engine sin tipos de FFmpeg	De convención: `GraphRunner.h` y `MediaDecoder.h` son internos, y el resto de FFmpeg se incluye desde los `.cpp`, a través de `ffmpeg_includes.h`.
View sin FFmpeg	**De convención:** VideoEditor importa `FFmpeg.props` para enlazar y copiar las DLL, así que el compilador sí encontraría esos headers.
View sin escribir en el modelo	Del tipo: recibe el proyecto como `const` a través de `project()`.

Decisiones de diseño principales

- **Tiempo en microsegundos enteros** (`TimeUs`, 64 bits): los cortes y transiciones no acumulan errores de redondeo. Solo el engine convierte a las bases de tiempo de FFmpeg.
- **Una sola cadena de filtros para preview y exportación:** lo que se ve es lo que se exporta.
- **Instantáneas por valor** (`TimelineSnapshot`): los hilos de trabajo reciben una copia, así que el modelo no necesita mutex.
- **Errores sin excepciones en la edición:** las operaciones devuelven `OpResult` y, si fallan, no cambian nada.

- **Texto en UTF-8 en todas las capas:** la conversión a UTF-16 ocurre solo en `src/VideoEditor/Utf8.cpp`, y así funcionan las rutas con tildes o ñ.

5. Estructura del repositorio

```
VideoEditor.slnx            Solución de Visual Studio
props/                      Common.props (C++20, /W4, salida) y FFmpeg.props
src/
  EditorModel/              Project, Timeline, Track, Clip, Effect, Transition,
serializador...
  EditorEngine/             Probe, decoder, filtros, reproducción, exportación
  EditorController/         EditorController
  VideoEditor/              Diálogos, timeline, preview y recursos (.rc)
tests/EditorTests/          Tests y MediosPrueba
third_party/                doctest y nlohmann/json
tools/                      generar_medios_prueba.ps1
openspec/                   Specs vigentes y el change archivado
bin/, obj/                  Salidas de compilación (no versionadas)
```

6. Capa Model

C++ estándar y `nlohmann::json`; no conoce FFmpeg, MFC ni Windows.

```
Project
├── MediaPool ── MediaInfo (ruta, duración, streams, códecs, offline)
├── CanvasSettings (ancho, alto, fps)
├── ExportSettings
└── Timeline
    └── Track (video o audio; muted; videoTrackId)
        ├── Clip (mediaId, sourceIn, sourceOut, timelineStart, linkedClipId)
        │   ├── Effect[] (variant: Grayscale, BrightnessContrast, Blur, Fade)
        │   └── PipTransform (x, y, scale, opacity)
        └── Transition (fromClipId, toClipId, style, duration)
```

Conceptos clave

- **Clip:** referencia al tramo `[sourceIn, sourceOut)` de un archivo, colocado en `timelineStart`. No contiene datos del medio.
- **Orden de las pistas:** primero las de video (`V1` es el fondo y las siguientes van encima), después las de audio de video y al final las generales.
- **Clips vinculados:** un video con audio crea dos clips enlazados por `linkedClipId` en ambos sentidos. Las operaciones se aplican a los dos hasta que se desvinculan.
- **Transición:** une dos clips consecutivos de una pista de video que se solapan durante `duration`. Se identifica por su clip de salida; si los dos clips tienen audio vinculado, hay una transición espejo entre los audios.

Reglas de `Project`

Todas las modificaciones pasan por `Project`, que sigue siempre estos pasos:

1. Valida la operación y, si no es válida, devuelve `OpResult::fallo` sin cambiar nada.
2. Aplica el cambio manteniendo los invariantes: clips ordenados y sin solapes (salvo en una transición), y al menos una pista de video.
3. Incrementa `revision()` y marca el proyecto como modificado (`dirty`).

Rangos de los parámetros

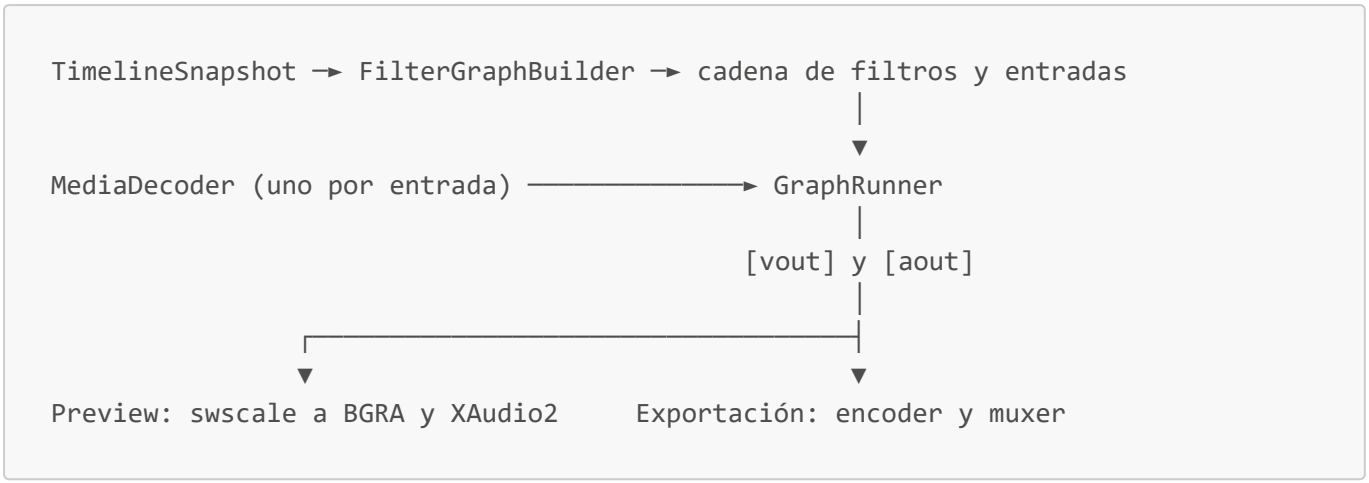
Parámetro	Rango	Fuera de rango
Brillo / contraste	-1..1 / 0..3	Se limita
Radio de desenfoque	0..50	Se limita
PiP: `x`, `y`, opacidad / escala	0..1 / 0.05..1	Se limita
Fade in + fade out	Hasta la duración del clip	Se rechaza
Duración de una transición	De un cuadro a `maxTransitionDuration`	Se rechaza
Duración de un clip tras un trim	Al menos un cuadro	Se rechaza

7. Capa Engine

Es la única capa que llama a FFmpeg. Usa además XAudio2 para el sonido del preview.

Clase	Visibilidad	Función
`MediaProbe`	Pública	Abre un archivo en solo lectura y devuelve `MediaInfo` o un error legible
`FilterGraphBuilder`	Pública	Convierte un `TimelineSnapshot` en una cadena de filtros
`PlaybackEngine`	Pública	Reproduce en un hilo propio y entrega cuadros BGRA
`ExportJob`	Pública	Exporta en un hilo propio con progreso y cancelación
`EncoderCatalog`	Pública	Prueba qué encoders abren en el equipo y cuáles admite cada contenedor
`FFmpegError`	Pública	Traduce códigos `AVERROR` a texto
`MediaDecoder`	Interna	Decodifica un stream limitado a un rango, con seek preciso
`GraphRunner`	Interna	Ejecuta el grafo y alimenta sus entradas a demanda
`AudioOutput`	Interna	Salida XAudio2 y reloj maestro del preview

Del timeline al cuadro



Cómo se construye el grafo

- **Pistas:** cada una es un `concat` de segmentos (clips y huecos) que cubren todo el rango pedido.
- **Rejilla de cuadros:** los segmentos de video se cuantizan a los cuadros del proyecto, para que las pistas no se desfasen.
- **Huecos:** negro en `V1`, transparente en las pistas superiores y silencio (`anullsrc`) en audio.

Elemento del modelo	Filtro de libavfilter
Escala de grises	`hue=s=0`
Brillo/contraste	`lutyuv` sobre la luminancia
Desenfoque	`gblur`
Fade in/out	`fade`
Transición de video	`xfade` (`fade`, `wipeleft`, `wiperight`, `slideleft`, `sliderright`, `circleopen`)
Transición de audio	`acrossfade`
Pistas superiores y PiP	`overlay` con `eof_action=pass`
Mezcla de audio	`amix` con `normalize=0`

Ejemplo real, generado por los tests, con dos clips seguidos en `V1` y el audio del primero:

```
[in0]setpts=PTS-
STARTPTS,scale=320:240:force_original_aspect_ratio=decrease:force_divisible_by=2,p
ad=320:240:(ow-iw)/2:(oh-
ih)/2:color=black,setsar=1,fps=25/1,format=yuv420p,tpad=stop_mode=clone:stop_durat
ion=1,trim=end_frame=125,setpts=PTS-STARTPTS[v1s0];
[in1]setpts=PTS-
STARTPTS,scale=320:240:force_original_aspect_ratio=decrease:force_divisible_by=2,p
ad=320:240:(ow-iw)/2:(oh-
ih)/2:color=black,setsar=1,fps=25/1,format=yuv420p,tpad=stop_mode=clone:stop_durat
ion=1,trim=end_frame=75,setpts=PTS-STARTPTS[v1s1];
[v1s0][v1s1]concat=n=2:v=1:a=0,setpts=round(PTS-STARTPTS+0.000000/TB)[vout];
[in2]atrim=start=0.000000:end=5.000000,asetpts=PTS-
STARTPTS,aresample=48000,aformat=sample_fmts=fltp:sample_rates=48000:channel_layou
ts=stereo,apad=whole_dur=5.000000,atrim=duration=5.000000[a0s0];
anullsrc=r=48000:cl=stereo,aformat=sample_fmts=fltp:sample_rates=48000:channel_lay
outs=stereo,atrim=duration=3.000000[a0s1];
[a0s0][a0s1]concat=n=2:v=0:a=1[at0];
[at0]anull,asetpts=round(PTS-
STARTPTS+0.000000/TB),aformat=sample_fmts=flt:sample_rates=48000:channel_layouts=s
tereo[aout]
```

Para generarlo:

```
bin\Debug\EditorTests.exe -tc="Imprimir grafo de ejemplo" --no-skip
```

Encoders

Tipo	Candidatos	Observación
Video	`h264_mf`, `hevc_mf`	MediaFoundation: dependen del hardware y respetan el bitrate de forma aproximada
Video	`mpeg4`	Por software; es la opción por defecto
Audio	`aac`, `aac_mf`	`aac` es la opción por defecto

`EncoderCatalog` prueba cada candidato con un `avcodec_open2` de 64x64, una sola vez, y consulta la compatibilidad con `avformat_query_codec`. En el equipo de desarrollo, 10 s a 720p con `h264_mf` y 2000 kbps pedidos dieron 2053 kbps de media (+2.6 %); se considera aceptable hasta un ±10 %.

8. Capa Controller

`controller::EditorController` es la única puerta de la View hacia el modelo y el engine:

- **Lectura:** `project()`, `seleccion()` y `playhead()`.
- **Edición:** importar, pistas, mover, dividir, recortar, eliminar, desvincular, efectos, PiP y transiciones.
- **Estado de edición (no se guarda en el proyecto):** selección de pista, clips o transición, cursor y ruta del proyecto.

- **Archivo:** `guardar`, `abrir` y `nuevoProyecto`.
- **Exportación:** `validarExportacion`, `startExport`, `cancelExport` y consulta del progreso.
- **Preview:** `iniciarPreview`, `sincronizarPreview`, `reproducirPausar` y `actualizarPreview`.

Dos comportamientos importantes:

- **Acciones sobre la selección, "todo o nada":** dividir o eliminar varios clips restaura el proyecto si alguna operación falla.
- **Abrir un proyecto no destruye el actual:** el archivo se carga en un `Project` temporal y solo lo reemplaza si es válido.

9. Capa View

Clase	Función
`CVideoEditorApp`	Registra las clases de ventana, crea el controller y abre el editor
`CEditorDlg`	Ventana principal: menús, atajos, temporizadores y coordinación de los paneles
`CTimelineWnd`	Timeline dibujado a mano, con todos los gestos del mouse
`CPreviewWnd`	Pinta el cuadro actual del preview
`EscalaTiempo`	Conversión tiempo↔píxel, zoom e imán; **sin MFC**, para poder probarla
`CPanelClipDlg`, `CPanelTransicionDlg`, `CPanelMedioDlg`	Paneles de propiedades
`CExportarDlg`	Diálogo de exportación
`Utf8`	Conversión `CString` ↔ UTF-8

Los diálogos se editan en `VideoEditor.rc` con el Editor de recursos. Ese archivo está en UTF-16.

Los paneles y el timeline piden cambios al controller y avisan a `CEditorDlg` con `WM_APP_VISTA_CAMBIO`, y este actualiza el resto de la interfaz. Los cambios del modelo llegan al preview 100 ms después del último, para agrupar los movimientos seguidos de un slider.

10. Hilos y concurrencia

Hilo	Hace	No hace
Interfaz (MFC)	Mensajes, ventanas, GDI y modelo	Decodificar o codificar
Render (`PlaybackEngine`)	Construye el grafo, decodifica y envía audio	Tocar ventanas o el modelo vivo
Exportación (`ExportJob`)	Filtra, codifica y escribe	Tocar ventanas o el modelo vivo

- **Comunicación hacia la interfaz:** los hilos de trabajo solo usan `PostMessage` (`WM_APP_PREVIEW_AVISO`, `WM_APP_EXPORT_DONE`), y la ventana consulta el estado en su handler.
- **Órdenes al preview:** política "la última gana"; un seek o timeline nuevo reemplaza al pendiente.
- **Sincronía:** el reloj maestro son las muestras que XAudio2 reprodujo realmente. La interfaz muestra el último cuadro con `pts <= reloj` y descarta los atrasados: si el equipo no da abasto, el video salta pero sigue sincronizado. Sin dispositivo de audio se usa un reloj de pared.
- **Búferes del preview:** cola de 3 cuadros, hasta 6 bloques de audio de 20 ms y ancho máximo de 960 px.
- **Cancelación:** `std::stop_token`, revisado entre cuadros. Los `std::jthread` se detienen y esperan al destruirse.

11. Formato del proyecto (JSON)

Versión actual: `formatVersion: 2`. Es texto legible, con los tiempos en microsegundos (sufijo `Us`) y los medios referenciados por ruta absoluta; nunca se copian.

```
{
  "formatVersion": 2,
  "canvas": {
    "width": 1920, "height": 1080,
    "fps": { "num": 30, "den": 1 }
  },
  "media": [
    {
      "id": 1, "path": "C:/videos/toma.mp4", "durationUs": 5000000,
      "hasVideo": true, "width": 1920, "height": 1080,
      "fps": { "num": 30, "den": 1 }, "videoCodec": "h264",
      "hasAudio": true, "sampleRate": 48000, "channels": 2,
      "audioCodec": "aac"
    }
  ],
  "tracks": [
    {
      "id": 2, "kind": "video", "name": "V1",
      "muted": false, "videoTrackId": 0,
      "clips": [
        {
          "id": 5, "mediaId": 1, "linkedClipId": 6,
          "sourceInUs": 0, "sourceOutUs": 5000000, "startUs": 0,
          "effects": [ { "type": "blur", "radius": 5.0 } ],
          "pip": { "x": 0.0, "y": 0.0, "scale": 1.0, "opacity": 1.0 }
        }
      ],
      "transitions": []
    }
  ],
  "export": {
    "container": "mp4", "videoCodec": "mpeg4", "audioCodec": "aac",
    "videoBitrateKbps": 5000, "audioBitrateKbps": 192,
    "preset": "1080p", "width": 1920, "height": 1080, "outputPath": ""
  }
}
```

El ejemplo está abreviado e ilustra la forma del archivo: un proyecto real también incluye el clip vinculado `6` y las pistas `AV1` y `A1`.

- **Carga "todo o nada":** se valida el archivo entero (campos, rangos, referencias, vínculos y transiciones). Si algo falla, el proyecto abierto no cambia.
- **Medios que faltan:** el controller vuelve a analizar cada medio. Los que no abren quedan *offline*: sus clips se ven en el timeline, pero se renderizan en negro y en silencio, y se informa al usuario.
- **Versión 1:** se puede seguir abriendo. La transformación no se aplicaba en `V1`, así que al cargar esos clips ocupan todo el lienzo.

12. Pruebas

El proyecto `EditorTests` usa doctest y prueba todas las capas salvo la parte MFC de la View. Hay **183 casos**: 181 se ejecutan por defecto y 2 son utilidades que se omiten salvo con `--no-skip`.

```
bin\Debug\EditorTests.exe                  Ejecutar todo
bin\Debug\EditorTests.exe -tc="*Transici*" Filtrar por nombre
bin\Debug\EditorTests.exe --help           Ver todas las opciones
```

Archivo	Casos	Cubre
`TestAudio.cpp`	29	Pistas de audio, vínculos y mezcla
`TestTransiciones.cpp`	24	Transiciones en el modelo y en el grafo
`TestCortes.cpp`	17	Split, trim y eliminar
`TestRender.cpp`	17 (1 omitido)	Grafo de filtros y render
`TestEfectos.cpp`	15	Efectos
`TestSeleccion.cpp`	14	Selección múltiple y sobrescritura
`TestPip.cpp`	13	Composición y PiP
`TestTimelineEditing.cpp`	11	Operaciones básicas del timeline
`TestEscalaTiempo.cpp`	10	Conversión tiempo↔píxel e imán
`TestExport.cpp`	10 (1 omitido)	Exportación
`TestPersistencia.cpp`	9	Guardar y cargar JSON
`TestMediaProbe.cpp`	6	Análisis y rechazo de archivos
`TestController.cpp`	4	Controller
`TestFFmpegError.cpp`	2	Mensajes de error
`TestTiempo.cpp`	2	Tiempo y formato

Medios de prueba: antes de compilar los tests, `tools/generar_medios_prueba.ps1` crea en `bin\<Configuración>\medios_prueba\` los archivos que falten: videos, tonos de audio, `sincronia.mp4` y un `falso.mp4` para probar el rechazo. Necesita `ffmpeg.exe` en la carpeta `bin\` de `FFmpegRoot`.

Pruebas manuales: el preview y la interfaz MFC no tienen tests automáticos. Estas comprobaciones se hacen a mano:

- **Sincronía de audio y video:** `bin\Debug\medios_prueba\sincronia.mp4` muestra un destello blanco y emite un pitido al mismo tiempo, una vez por segundo. Al importarlo y reproducirlo en el editor, el pitido debe oírse justo cuando la pantalla se pone blanca.
- **Bitrate de `h264_mf`:** este comando genera un video de prueba, y `ffprobe` permite medirlo:

```
bin\Debug\EditorTests.exe -tc="Medir h264_mf" --no-skip
ffprobe -v error -select_streams v:0 -show_entries
stream=bit_rate,avg_frame_rate,duration ^
        %TEMP%\editor_tests\h264_mf_2000.mp4
```

- **Grafo de filtros con la línea de comandos:** guarda la cadena en un archivo, reemplaza cada entrada `[inN]` por el stream del archivo correspondiente (`[0:v]` , `[0:a]` , `[1:v]` ...) y usa `-/filter_complex` , que en FFmpeg 9 reemplaza a `-filter_complex_script` :

```
ffmpeg -i a.mp4 -i b.mp4 -/filter_complex grafo.txt -map "[vout]" -map "
[aout]" salida.mp4
```

> **Último resultado:** el 2026-11-04, `bin\Debug\EditorTests.exe` terminó con 181 casos correctos, 0 fallidos y 2 omitidos (3244 aserciones). Se usó el binario ya compilado, sin recompilar; la cantidad de casos coincide con los fuentes actuales.

13. Especificaciones (OpenSpec)

Los requisitos se gestionan con OpenSpec, con un esquema dirigido por specs. En total hay 59 requisitos:

Capability	Requisitos	Tema
`timeline-editing`	12	Timeline y operaciones sobre clips
`audio-tracks`	8	Pistas de audio, vínculos y mezcla
`video-export`	8	Exportación
`clip-transitions`	6	Transiciones
`timeline-preview`	6	Preview
`video-effects`	6	Efectos
`project-persistence`	5	Guardar y cargar
`media-import`	4	Importación
`video-compositing`	4	Composición y PiP

El desarrollo se planificó en un único change, ya archivado, con la propuesta, el diseño y sus 55 tareas, todas completadas.

14. Limitaciones conocidas

- **Sin deshacer/rehacer:** la aplicación no ofrece esta función.
- **Solo Windows x64:** depende de MFC, XAudio2 y MediaFoundation.
- **Sin webm en la exportación:** el build de FFmpeg no trae un encoder VP8/VP9/AV1 utilizable. Tampoco incluye `libx264` , así que H.264 depende de `h264_mf` y del equipo.

- **Posible saturación de audio:** `amix` usa `normalize=0` y no hay ganancia por pista, así que varias pistas fuertes pueden saturar.
- **Sin efectos de audio ni de velocidad:** los efectos son solo de video, y las transiciones solo se aplican entre clips consecutivos de una misma pista.
- **El grafo se reconstruye en cada seek o edición:** se mitiga con el debounce, la política "la última gana" y la resolución reducida, pero puede notarse con muchos clips.
- **Rutas absolutas en el proyecto:** al mover los medios, sus clips quedan offline.
- **Ruta de FFmpeg por defecto específica del equipo de desarrollo:** en otro equipo hay que ajustar `FFmpegRoot`.